JAVA INTERVIEW PREPARATION

CHAPTER 68

HEAPS, PRIORITY QUEUES,
AND SELECTION ALGORITHMS

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

Heaps, Priority Queues, and Selection Algorithms

Senior / Lead Java Developer Reading Chapter

A priority queue answers a focused question: which currently available item has the highest scheduling priority? A binary heap is the usual in-memory implementation because it keeps the best item at the root while avoiding the cost of maintaining a completely sorted collection. This distinction matters. A heap is partially ordered, not sorted; it is excellent for repeated minimum or maximum removal, but poor for arbitrary lookup and ordered iteration.

Senior interviews use heaps to test more than index arithmetic. A strong candidate identifies the service contract behind the ordering, handles comparator equality and mutable priorities, distinguishes worst-case from amortized work, and recognizes when a heap-based design introduces starvation, stale entries, or concurrency problems.

Priority queue contract before representation

Suppose five jobs have priorities 2, 8, 4, 1, and 6. A maximum-priority queue promises that `8` is removed first. It does not promise that the remaining jobs are stored or iterated as `6, 4, 2, 1`.

```
Logical contract                One valid max-heap shape

peek -> 8                             8
remove -> 8                         /   \
next peek -> 6                      6     4
                                   / \
                                  1   2

Only parent-versus-child order is guaranteed.
```

A min-priority queue reverses the relation. Java's `PriorityQueue` is a min-heap under natural ordering or the supplied comparator: `peek()` exposes the least element. A maximum queue is normally created with a reversed comparator.

```
PriorityQueue<Integer> minQueue = new PriorityQueue<>();
PriorityQueue<Integer> maxQueue =
        new PriorityQueue<>(Comparator.reverseOrder());
```

The queue does not permit `null`. Its iterator has no sorted-order guarantee. `offer` and `poll` are O(log n) in the normal heap model, while `peek` is O(1). Removing or finding an arbitrary object is linear because a heap does not determine which branch contains that value.

Binary heap array model

A complete binary tree fills levels from left to right. That shape maps compactly to an array without node objects.

```
array index:   0   1   2   3   4   5   6
value:       [3] [7] [5] [9] [8] [6] [10]

                    3
                  /    \
                 7      5
               /  \    / \
              9    8  6  10
```

For zero-based index `i`:

```
parent(i) = (i - 1) / 2       when i > 0
left(i)   = 2 * i + 1
right(i)  = 2 * i + 2
```

In a min-heap, every parent compares less than or equal to its children. The invariant applies only along parent-child edges. Siblings have no required relationship, and a node need not compare with cousins on the same level.

The array representation gives good locality and low allocation overhead. Capacity may still grow by copying, and elements are references when the heap stores objects. A custom implementation must guard index arithmetic for extreme capacities, although practical Java arrays hit memory limits first.

Insertion uses sift-up

To insert, place the element in the next free array position. The complete-tree shape remains valid, but the new value may violate the heap property with its parent. Repeatedly exchange it upward until the parent is no worse.

```
insert 2 into min-heap

before repair:                    after sift-up:

        3                                 2
      /   \                             /   \
     7     5          ->               3     5
    / \   /                           / \   /
   9  8  6                           9  8  6
  /
 2
```

```
private void siftUp(int index) {
    int value = elements[index];
    while (index > 0) {
        int parent = (index - 1) >>> 1;
        if (elements[parent] <= value) break;
        elements[index] = elements[parent];
        index = parent;
    }
    elements[index] = value;
}
```

Moving parents down and writing the inserted value once can reduce assignments compared with swapping at every step. The loop invariant is that every heap edge is valid except possibly the edge connecting the moving hole to its parent. Height is O(log n), so insertion is O(log n).

Removal uses sift-down

Removing the root creates a hole. Move the last element to the root, reduce logical size, then compare it with the better of its children. In a min-heap, the smaller child must be selected; swapping with the larger child can leave the smaller child above a larger parent and violate the invariant.

```
private int removeMin() {
    if (size == 0) throw new NoSuchElementException();
    int result = elements[0];
    int moved = elements[--size];

    int index = 0;
    int half = size >>> 1; // nodes below half have no children
    while (index < half) {
        int left = (index << 1) + 1;
        int right = left + 1;
        int better = right < size && elements[right] < elements[left]
                ? right : left;
        if (moved <= elements[better]) break;
        elements[index] = elements[better];
        index = better;
    }
    if (size > 0) elements[index] = moved;
    return result;
}
```

A generic implementation must clear the old last slot to avoid retaining its object. It must also use the same comparator for every decision. If comparison throws during repair, the data structure may be left partially modified; library implementations specify behavior, while custom production structures need clear failure assumptions around comparators.

Building a heap is linear

Inserting `n` elements one at a time costs O(n log n), but an existing array can be heapified bottom-up in O(n). Begin with the last internal node and sift each node down.

```
for (int i = (size >>> 1) - 1; i >= 0; i--) {
    siftDown(i);
}
```

The linear result initially surprises people because an individual sift can take logarithmic time. Most nodes are near the leaves and move zero or one level. Only a small number near the root can travel far. Summing the work by node height yields a geometric series bounded by O(n).

```
many nodes:   height 0 -> no work
half as many: height 1 -> at most 1 step
quarter:      height 2 -> at most 2 steps
...
total work remains proportional to n
```

This is the preferred construction when all input is available. Repeated insertion is appropriate when elements arrive online.

Comparator correctness and stable priority

A comparator defines what the root means. Avoid subtraction such as `a.priority() - b.priority()` because integer overflow can reverse the result. Compose safe comparisons and add deterministic tie-breakers when equal priority requires FIFO behavior.

```
record Job(long sequence, int priority, Runnable task) {}

Comparator<Job> order = Comparator
        .comparingInt(Job::priority).reversed()
        .thenComparingLong(Job::sequence);
```

Without the sequence, equal-priority jobs may leave in any order. This is not a heap bug; stability was never part of the contract. The sequence must be monotonic within the relevant scheduler and must not overflow

unnoticed in a long-lived service.

Fields used by the comparator must not change while an element is in the queue. If a job's priority changes in place, the heap does not automatically reposition it. Remove and reinsert the element, use an indexed heap with an explicit change-key operation, or insert a new immutable entry and treat the old one as stale.

An indexed heap adds a map from logical ID to current array position. A swap must update both array positions and both map entries as one invariant. `decreaseKey`, `increaseKey`, and arbitrary removal can then repair upward or downward in O(log n). The benefit is precise update and cancellation; the cost is extra memory and more mutation points. If duplicate logical IDs are allowed, a one-position map is insufficient and the identity contract must change.

```
heap array:       [job C, job A, job B]
position map:      C -> 0, A -> 1, B -> 2

every swap must update both representations
```

Comparator transitivity matters. If A precedes B, B precedes C, but C precedes A, no tree arrangement can satisfy the relation consistently. Such defects can produce unstable results or exceptions in ordering algorithms.

Top-k selection

To retain the `k` largest elements from a stream, maintain a min-heap of at most `k` candidates. The root is the weakest retained candidate.

```
static List<Integer> largest(int[] values, int k) {
    if (k < 0 || k > values.length) {
        throw new IllegalArgumentException("k");
    }
    PriorityQueue<Integer> best = new PriorityQueue<>(k);

    for (int value : values) {
        if (best.size() < k) {
            best.offer(value);
        } else if (k > 0 && value > best.peek()) {
            best.poll();
            best.offer(value);
        }
    }
    return new ArrayList<>(best); // not sorted
}
```

Time is O(n log k), and additional space is O(k). Returning the queue's iteration order as a ranked result is wrong; sort the retained elements afterward when output order matters. For the `k` smallest values, use a max-heap of size `k`.

If input is an in-memory mutable array and only one kth element is needed, quickselect often offers expected O(n) time and O(1) extra array space. A heap is preferable for streaming input, small `k`, non-mutating requirements, or repeated access to the selected frontier.

Merging sorted sequences

To merge `m` sorted sequences, place the first element of each nonempty sequence in a min-heap. Remove the smallest, emit it, then insert the next element from the same sequence.

```
record Cursor(int value, int listIndex, int elementIndex) {}

static List<Integer> merge(List<List<Integer>> lists) {
    PriorityQueue<Cursor> heap = new PriorityQueue<>(
            Comparator.comparingInt(Cursor::value));
    for (int i = 0; i < lists.size(); i++) {
        if (!lists.get(i).isEmpty()) {
            heap.offer(new Cursor(lists.get(i).get(0), i, 0));
        }
    }

    List<Integer> result = new ArrayList<>();
    while (!heap.isEmpty()) {
        Cursor current = heap.poll();
        result.add(current.value());
        int next = current.elementIndex() + 1;
        List<Integer> source = lists.get(current.listIndex());
        if (next < source.size()) {
            heap.offer(new Cursor(source.get(next),
                    current.listIndex(), next));
        }
    }
    return result;
}
```

For `N` total elements, time is O(N log m), and heap space is O(m). Production implementations often stream output instead of collecting it, close input resources, and define what happens when one source fails halfway through.

Running median with two heaps

A running median can be maintained with a max-heap for the lower half and a min-heap for the upper half.

```
lower max-heap      upper min-heap
 values <= median   values >= median

       7                 9
     /   \             /   \
    3     5           12   15

sizes differ by at most one
```

Insert according to the lower root, then rebalance so sizes differ by at most one. If both sizes are equal, the median is the average of the two roots; add as `long` or divide separately to avoid integer overflow. If exact decimal behavior matters, define rounding and numeric type.

Deletion of arbitrary old values, as required for a sliding-window median, is harder. Java's `PriorityQueue.remove(Object)` is linear. Common solutions use two ordered multisets, an indexed heap, or lazy-deletion maps that count entries to discard when they eventually reach a root. Lazy deletion requires careful logical-size accounting because physical heap size includes stale entries.

Heap sort and in-place ordering

Heap sort builds a max-heap, exchanges the root with the end, shrinks the heap region, and restores the heap. It guarantees O(n log n) time and uses O(1) auxiliary array space, but it is not stable and often has poorer cache behavior than tuned library sorts.

```
[ heap region | sorted suffix ]
       ^             grows leftward
```

The practical lesson is not to reimplement sorting in application code. Heap sort is valuable for understanding the invariant and for environments where worst-case time plus low auxiliary space dominate stability and

locality.

Scheduling, fairness, and stale work

Priority queues appear in deadline schedulers, timers, network simulation, best-first search, and resource allocation. A pure priority policy can starve low-priority jobs when high-priority work arrives continuously. Aging, quotas, weighted fair queues, or separate classes of service may be needed.

Capacity must be separate from ordering. If eight workers complete 20 jobs per second each and the queueing budget is 250 milliseconds, a rough steady-state allowance is about `160 * 0.25 = 40` jobs, plus a deliberately measured burst margin. An unbounded heap of 100,000 jobs would permit more than ten minutes of waiting at the same drain rate. Priority does not create throughput; it only decides who waits first.

Time-based scheduling must distinguish priority from readiness. The earliest deadline may be at the heap root, but it cannot execute before its due time. A scheduler waits for the root's delay, then rechecks because a newly inserted earlier deadline may need to wake it. Java's `DelayQueue` captures delayed eligibility, while `ScheduledThreadPoolExecutor` adds execution workers and lifecycle behavior.

Retries can dominate priority queues if every failure creates immediate high-priority work. Include next-attempt time, maximum attempts, jitter, and a dead-letter outcome. Queue length alone does not reveal stale work; monitor oldest age, lateness, service time, rejection, and the distribution of priorities.

Concurrency choices

`PriorityQueue` is not thread-safe. Protecting individual method calls with an external lock is sufficient only if compound workflows also use the same lock. `PriorityBlockingQueue` is thread-safe and blocking on removal, but it is unbounded: insertion does not provide backpressure. An unbounded priority queue can turn overload into memory growth while continuing to accept work.

Changing priority safely is especially difficult under concurrency. Remove-then-add creates a moment when the job is absent; in-place mutation corrupts ordering; duplicate immutable entries require version checks. Define one owner thread, lock the complete mutation, or use a scheduler designed around immutable commands.

Cancellation with immutable versioned entries can be implemented by recording the latest generation per job ID. Polling discards any entry whose generation is no longer current. This avoids arbitrary heap removal but permits stale entries to consume memory until they reach the root. A compaction or rebuild threshold may be necessary when cancellation is frequent and deadlines are far in the future.

Testing heap behavior

Test empty and one-element heaps, ascending and descending input, duplicates, all-equal values, comparator tie-breakers, integer extremes, and repeated drain-to-empty cycles. Verify properties rather than only examples:

- the root is globally minimal or maximal;
- every parent satisfies the heap relation with existing children;
- draining produces ordered output;
- heapify contains the same multiset as input;
- top-k matches a fully sorted oracle on small randomized arrays;
- mutable-priority behavior is rejected or explicitly repaired.

For custom heaps, test capacity growth, reference clearing, comparator exceptions, and large indices. For concurrent schedulers, test cancellation, shutdown, duplicate commands, starvation controls, and overload.

Common senior-level traps

- Calling a heap a sorted tree.
- Assuming `PriorityQueue` iteration is priority order.
- Using a max-heap for top-k largest and thereby retaining the wrong boundary.
- Claiming heap construction is O(n log n) when bottom-up heapify is O(n).
- Mutating comparator fields while an element remains queued.
- Forgetting deterministic tie-breakers when equal priority needs FIFO.
- Using subtraction in comparators and overflowing.
- Treating `PriorityBlockingQueue` as bounded backpressure.
- Using linear arbitrary removal inside an algorithm claimed to be logarithmic.
- Ignoring starvation, cancellation, stale entries, and shutdown in a scheduler.

A strong interview answer

I first define whether the queue exposes the minimum or maximum and what equality means. A binary heap stores a complete tree in an array and maintains parent-child order, giving O(1) peek and O(log n) insertion and root removal. It does not provide sorted iteration or logarithmic arbitrary search. Bottom-up construction is O(n). For top-k streaming selection I keep the opposite heap of size k so the weakest retained candidate is at the root. In production I make priority immutable, add stable tie-breakers if required, define starvation and overload policy, and choose a concurrent or bounded design based on the complete workflow rather than the interface name.

Chapter summary

Heaps maintain only enough order to expose the next priority efficiently. Their array shape explains logarithmic repair, linear bottom-up construction, and poor arbitrary search. Priority-queue algorithms work by keeping a small frontier: top-k candidates, one cursor per sorted source, two median halves, or the next scheduled event. Correct production use additionally requires stable ordering, immutable priorities, capacity, fairness, cancellation, and concurrency semantics.

Revision Notes

- Java `PriorityQueue` is a min-priority queue under its comparator.
- A binary heap is complete and partially ordered, not fully sorted.
- Zero-based children are `2i + 1` and `2i + 2`; parent is `(i - 1) / 2`.
- Insert at the end and sift up: O(log n).
- Remove the root, move the last element, and sift down: O(log n).
- Peek is O(1); arbitrary search or removal is O(n).
- Bottom-up heapify is O(n), not O(n log n).
- Queue iteration is not sorted; drain or copy-and-sort for ranked output.
- Comparator fields should be immutable while queued.
- Add a sequence tie-breaker when equal priority must preserve arrival order.
- Top-k largest uses a min-heap of size k: O(n log k).
- Merge m sorted streams with one cursor per stream: O(N log m).
- Two heaps maintain a running median; avoid overflow when averaging roots.
- Lazy deletion requires separate logical and physical size accounting.
- Heap sort is in-place O(n log n), but unstable.
- `PriorityBlockingQueue` is concurrent but unbounded.
- Production schedulers need capacity, fairness, retry, cancellation, and shutdown rules.

Likely interview question: Why is heapify O(n)?

Most nodes are leaves or near leaves and require little or no movement. Summing maximum sift distance by height produces a convergent geometric series proportional to n, even though the root alone may move O(log n) levels.

Likely interview question: How do you find the kth largest element?

For streaming or non-mutating input, keep a min-heap of size k and return its root after processing all values, for O(n log k) time and O(k) space. For one mutable in-memory array, quickselect may provide expected O(n) time.

Likely interview question: Why can changing an object's priority break the queue?

The heap placed the object using its earlier comparisons. Mutating the field does not trigger sift-up or sift-down, so the root may no longer be best. Reinsert, use a change-key structure, or queue immutable versioned entries.

Likely interview question: Is PriorityBlockingQueue backpressure?

No. It is thread-safe and supports blocking removal, but it is unbounded. Sustained overload can still increase memory and wait time. Backpressure requires bounded admission and an explicit rejection, blocking, timeout, or spill policy.